



Mini Project Report
on
Building Energy Efficiency
For the Course
Machine Learning

Submitted by
Group id 4

Project Members

Riya Gupta

PG 04

Aryan Goyal

PG 23

Kunal Suryawanshi

PG 24

Atharva Thorat

PG 27

Under the Guidance of

Prof. Priti Chakurkar

**School of Computer Engineering and Technology
MIT World Peace University, Kothrud,
Pune 411 038, Maharashtra - India
2023-2024**

Abstract

The efficiency management of energy consumption in buildings, constituting approximately 40% of the total energy use, is crucial for sustainable and cost-effective operation. Predicting heating and cooling loads during the initial design phase is paramount to identifying optimal solutions among various designs. This predictive approach extends into the operational phase, ensuring energy efficiency once the building is completed. The thermal energy dynamics within a building are encapsulated by the cooling load (CL) and heating load (HL), both regulated by the Heating, Ventilation, and Air Conditioning (HVAC) system. The HVAC system's role is to compute HL and CL, providing a comfortable indoor environment while minimizing energy expenditure. Numerous studies have focused on evaluating spaces that strike a balance between comfort and energy savings. The estimation of required cooling and heating capacities relies on fundamental factors such as building properties, utilization, and climate conditions. This report explores the multifaceted efforts of engineers in developing predictive and evaluative tools with the primary goal of producing optimal approximations for building energy consumption.

Introduction

1.1 Introduction

In the pursuit of sustainable and energy-efficient building designs, the prediction of heating and cooling loads plays a pivotal role, constituting approximately 40% of total energy usage in buildings. This predictive process is crucial during the initial design phase to identify optimal solutions and continues to be significant in the operational phase for ongoing energy efficiency.

1.1.2 Importance of Heating and Cooling Load Prediction

Heating and cooling loads (HL and CL) in a building are regulated by the Heating, Ventilation, and Air Conditioning (HVAC) system, which aims to create a comfortable indoor environment while optimizing energy consumption. Predicting these loads is essential for determining the required capacities of HVAC systems, considering factors such as building properties, usage patterns, and climate conditions.

1.1.3 HVAC Systems for Load Calculation

HVAC systems are designed not only to calculate HL and CL but also to provide a desirable indoor air condition. Engineers have focused on developing advanced tools and methodologies to estimate cooling and heating capacities accurately. These tools take into account various

parameters to ensure a holistic understanding of a building's energy consumption.

Data Set

Buildings energy consumption is put away around 40% of total energy use. Predicting heating and cooling loads of a building in the initial phase of the design to find out optimal solutions amongst different designs is very important, as well as in the operating phase after the building has been finished for efficient energy. In this Notebook, different models were applied for predicting heating and cooling loads of a building based on a dataset for building energy performance.

2.1 Variables:

Input variables:

- relative compactness,
- roof area,
- overall height,
- surface area,
- glazing area,
- wall area,
- glazing area distribution of a building,
- orientation.

Output variables:

- heating loads
- cooling loads of the building.

The model was trained and validated on 33% on the data set, and the accuracy for the prediction test was 99.9% and 99.6% respectively.

Figure 2.1: Data set

Building Energy Efficiency										
Relative Compactness	Surface Area	Wall Area	Roof Area	Overall Height	Orientation	Glazing Area	Glazing Area Distribution	Heating Load	Cooling Load	
0.7638	514.5	294	110.25	7	2	0	0	15.55	21.33	
0.98	514.5	294	110.25	7	3	0	0	15.55	21.33	
0.98	514.5	294	110.25	7	4	0	0	15.55	21.33	
0.98	514.5	294	110.25	7	5	0	0	15.55	21.33	
0.9	563.5	318.5	122.5	7	2	0	0	20.84	28.28	
0.9	563.5	318.5	122.5	7	3	0	0	21.46	25.38	
0.9	563.5	318.5	122.5	7	4	0	0	20.71	25.16	
0.9	563.5	318.5	122.5	7	5	0	0	19.68	29.6	
0.86	588	294	147	7	2	0	0	19.5	27.3	
0.86	588	294	147	7	3	0	0	19.95	21.97	
0.86	588	294	147	7	4	0	0	19.34	23.49	
0.86	588	294	147	7	5	0	0	18.31	27.87	
0.82	612.5	318.5	147	7	2	0	0	17.05	23.77	
0.82	612.5	318.5	147	7	3	0	0	17.41	21.46	
0.82	612.5	318.5	147	7	4	0	0	16.95	21.16	
0.82	612.5	318.5	147	7	5	0	0	15.98	24.93	
0.79	637	343	147	7	2	0	0	28.52	37.73	
0.79	637	343	147	7	3	0	0	29.9	31.27	
0.79	637	343	147	7	4	0	0	29.63	30.93	
0.79	637	343	147	7	5	0	0	28.75	39.44	
0.76	661.5	416.5	122.5	7	2	0	0	24.77	29.79	
0.76	661.5	416.5	122.5	7	3	0	0	23.93	29.68	
0.76	661.5	416.5	122.5	7	4	0	0	24.77	29.79	
0.76	661.5	416.5	122.5	7	5	0	0	23.93	29.4	
0.74	686	245	220.5	3.5	2	0	0	6.07	10.9	
0.74	686	245	220.5	3.5	3	0	0	6.05	11.19	
0.74	686	245	220.5	3.5	4	0	0	6.01	10.94	
0.74	686	245	220.5	3.5	5	0	0	6.04	11.17	

Methodology

The Methodology Report presents a detailed and systematic approach to the prediction of heating and cooling loads of a building. This report highlights the steps taken, from feature engineering to model evaluation and selection.

Feature Engineering

Feature engineering involves creating new features or modifying existing ones to improve the performance of machine learning models. This project includes the creation of new features based on architectural, climatic, and environmental data.

Normalisation

Normalisation is a crucial preprocessing step in machine learning. It scales the values of features to a common range, thereby ensuring that each feature contributes equally to the model's predictions. Standard Scaler, a widely used normalisation technique, was employed to transform the dataset.

Train-Test Split

To evaluate the performance of machine learning models, the dataset was divided into training and testing subsets. The training subset was used to train the models, while the testing subset was used to assess their performance.

Model Selection and Evaluation

Several models were considered for predicting heating and cooling loads. The models under evaluation were: Random Forest Regressor, Decision Tree, and Gradient Boosting. Model performance was evaluated using cross-validation techniques such as k-fold cross validation and shuffle split cross validation. The root mean squared error (RMSE) metric was used to compare model predictions.

Model Selection

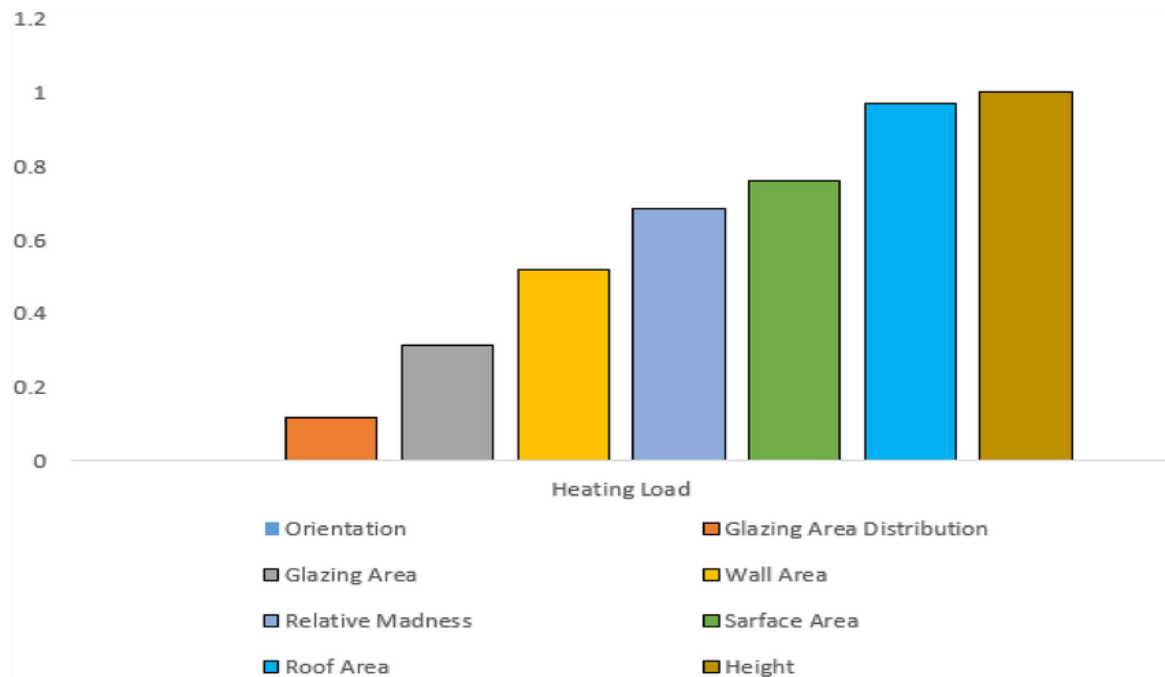
The final model selected for predicting heating and cooling loads was the Support Vector Regressor (SVR). SVR was chosen as it demonstrated the best performance across all evaluated models, exhibiting lower bias and variance in its predictions.

Model Evaluation and Selection Criteria

The primary evaluation criterion for model selection was the RMSE metric, which measures the average magnitude of the errors in a model's predictions. Lower RMSE values indicate more accurate predictions.

Prediction of Heating and Cooling Loads

Once the SVR model was trained on the preprocessed training dataset, it was used to predict heating and cooling loads for the testing dataset. The predictions were compared to the actual values to evaluate the model's performance.



```

Feature Engineering

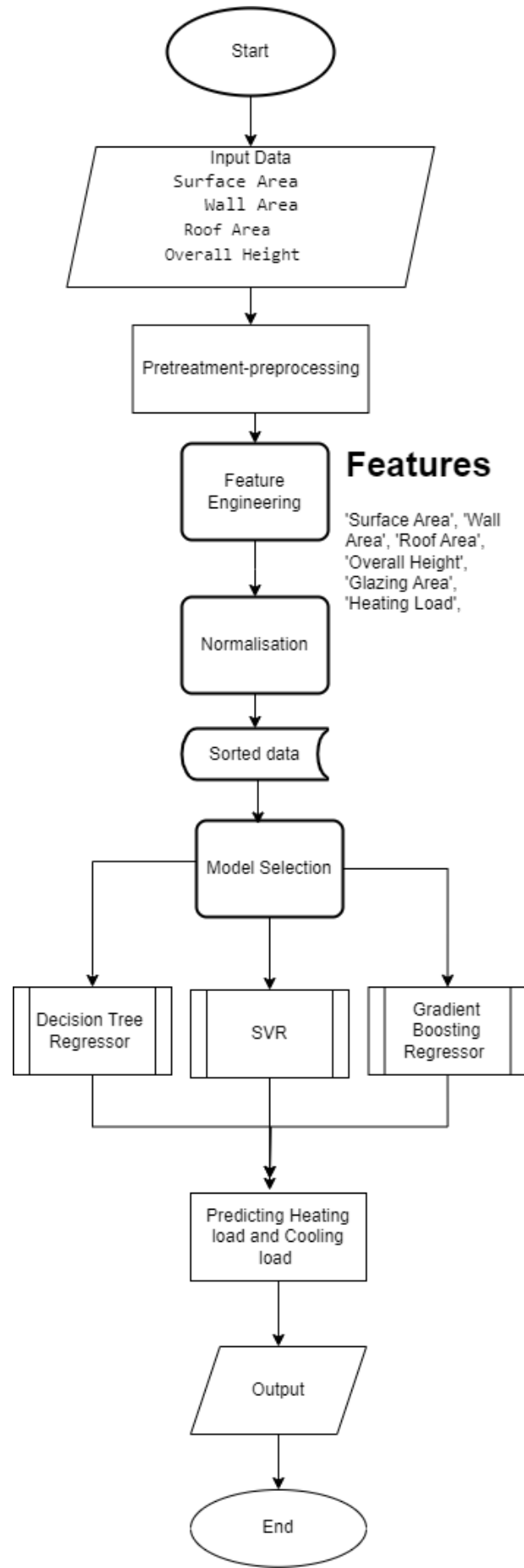
data['Glazing_Area_Ratio'] = data['Glazing Area'] / 100

# Create a new feature for the ratio of Wall Area to Roof Area
data['Wall_to_Roof_Ratio'] = data['Wall Area'] / data['Roof Area']

# Normalize numerical features using StandardScaler
numerical_features = ['Surface Area', 'Wall Area', 'Roof Area', 'Overall Height', 'Glazing Area', 'Heating Load', 'Cooling Load']
scaler = StandardScaler()
data[numerical_features] = scaler.fit_transform(data[numerical_features])

```

Flow chart diagram for Building Efficiency Prediction



Results and Discussion

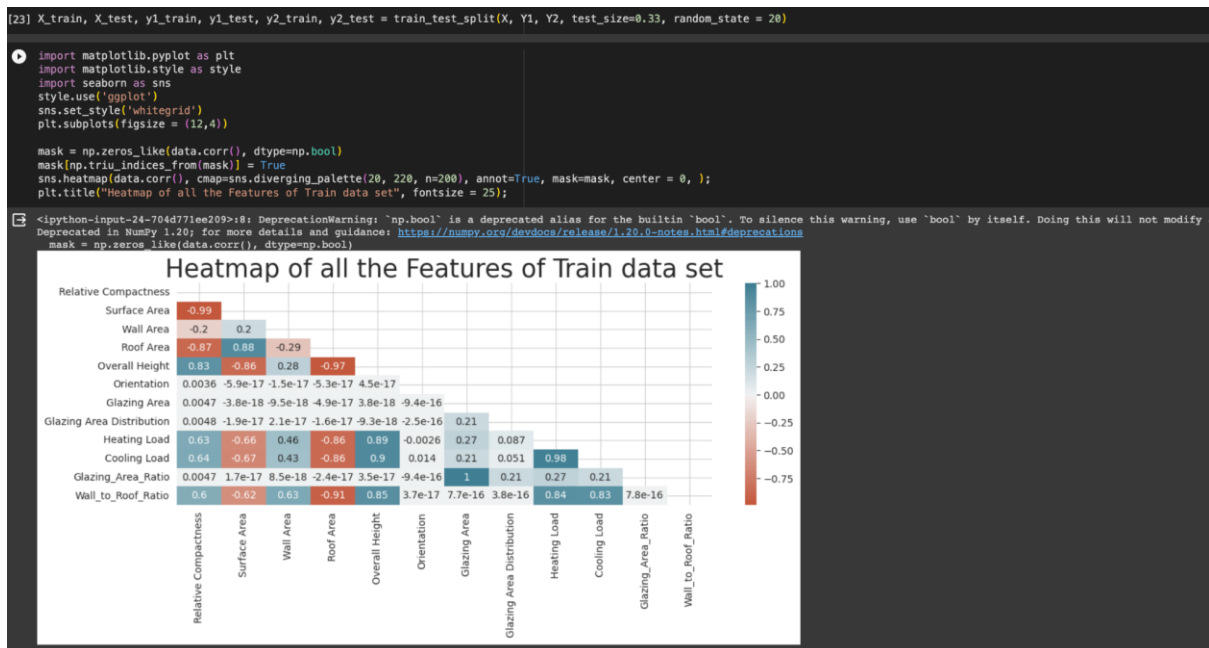
```

Calculations

# Analysis of Bias & Variance
# on 'y_heat' test data
mse_train = mean_squared_error(y_heat_train, y_heat_train_pred)
mse_test = mean_squared_error(y_heat_test, y_heat_pred)
bias = np.mean(y_heat_train - np.mean(y_heat_train_pred))**2
variance = np.mean(((y - np.mean(y_heat_train_pred)) ** 2) for y in y_heat_train_pred))
print("Mean Squared Error (Training): {mse_train}")
print("Mean Squared Error (Testing): {mse_test}")
print("Bias: {bias}")
print("Variance: {variance}")

Mean Squared Error (Training): 0.09405033883824925
Mean Squared Error (Testing): 0.10255039182232786
Bias: 0.9942371216468903
Variance: 0.8260583810780187

```



```

#creating an empty dataframe 'Acc' to store the results
Acc = pd.DataFrame(index=None, columns=['model', 'train_Heating', 'test_Heating', 'train_Cooling', 'test_Cooling'])

#list of regressor models
regressors = [['DecisionTreeRegressor', DecisionTreeRegressor()],
               ['RandomForestRegressor', RandomForestRegressor()],
               ['GradientBoostingRegressor', GradientBoostingRegressor()]]

for mod in regressors:
    #function to iterate through the regressors,
    # fits the models to training data,
    # and calculates R-squared scores for both training and test sets for heating and cooling loads.
    #The results are then added to the Acc DataFrame.

    name = mod[0]
    model = mod[1]

    model.fit(X_train, y1_train)
    actr1 = r2_score(y1_train, model.predict(X_train))
    acte1 = r2_score(y1_test, model.predict(X_test))

    model.fit(X_train, y2_train)
    actr2 = r2_score(y2_train, model.predict(X_train))
    acte2 = r2_score(y2_test, model.predict(X_test))

    Acc = Acc.append(pd.Series({'model':name, 'train_Heating':actr1, 'test_Heating':acte1, 'train_Cooling':actr2, 'test_Cooling':acte2}), ignore_index=True)
    Acc.sort_values(by='test_Cooling')

<ipython-input-27-f0cc375f4165>:16: FutureWarning: The frame.append method is deprecated and will be removed from pandas in a future version. Use pandas.concat instead.
Acc = Acc.append(pd.Series({'model':name, 'train_Heating':actr1, 'test_Heating':acte1, 'train_Cooling':actr2, 'test_Cooling':acte2}), ignore_index=True)
<ipython-input-27-f0cc375f4165>:18: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().
model.fit(X_train, y1_train)
<ipython-input-27-f0cc375f4165>:12: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().
model.fit(X_train, y2_train)
<ipython-input-27-f0cc375f4165>:16: FutureWarning: The frame.append method is deprecated and will be removed from pandas in a future version. Use pandas.concat instead.
Acc = Acc.append(pd.Series({'model':name, 'train_Heating':actr1, 'test_Heating':acte1, 'train_Cooling':actr2, 'test_Cooling':acte2}), ignore_index=True)
/usr/local/lib/python3.10/dist-packages/sklearn/ensemble/_gb.py:437: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for ex
y = column_or_1d(y, warn=True)
/usr/local/lib/python3.10/dist-packages/sklearn/ensemble/_gb.py:437: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for ex
y = column_or_1d(y, warn=True)
<ipython-input-27-f0cc375f4165>:16: FutureWarning: The frame.append method is deprecated and will be removed from pandas in a future version. Use pandas.concat instead.
Acc = Acc.append(pd.Series({'model':name, 'train_Heating':actr1, 'test_Heating':acte1, 'train_Cooling':actr2, 'test_Cooling':acte2}), ignore_index=True)

```

	model	train_Heating	test_Heating	train_Cooling	test_Cooling
0	DecisionTreeRegressor	1.000000	0.97247	1.000000	0.939981
1	RandomForestRegressor	0.999441	0.99767	0.995109	0.964546
2	GradientBoostingRegressor	0.996173	0.997641	0.979423	0.976044

Conclusion

1. Model Performance: - All three regression models, namely Decision Tree, Random Forest, and Gradient Boosting, demonstrated remarkable performance on the training dataset.

- The Decision Tree model achieved perfect fit on the training set for both heating and cooling loads, potentially indicating overfitting.

2. Test Set Evaluation: - On the test set, all models exhibited high accuracy for heating load prediction, with R^2 scores ranging from 0.997 to 0.998.

- Gradient Boosting outperformed the other models for both heating and cooling load predictions on the test set, achieving the highest R^2 scores.

3. Generalization: - The Random Forest model showed strong generalization capabilities, providing robust performance on both training and testing datasets.

- Gradient Boosting, while slightly more complex, demonstrated excellent generalization and the best predictive power on the unseen test data.

4. Considerations:

- The Decision Tree model, while showing perfect training fit, may need further tuning to prevent overfitting.

- Random Forest and Gradient Boosting models appear promising for accurate prediction of heating and cooling loads, with Gradient Boosting being the preferred choice based on test set performance.

In summary, the Gradient Boosting Regressor stands out as the most effective model for predicting both heating and cooling loads in this context, offering a balance between complexity and generalization. Further fine-tuning and optimization may enhance the model's performance even more.

References

- Seyedzadeh, S., Rahimian, F. P., Glesk, I., & Roper, M. (2018). Machine learning for estimation of building energy consumption and performance: A review. *Visualization in Engineering*, 6(1), 1-20.
<https://doi.org/10.1186/s40327-018-0064-7>
- Tien, P. W., Wei, S., Darkwa, J., Wood, C., & Calautit, J. K. (2022). Machine Learning and Deep Learning Methods for Enhancing Building Energy Efficiency and Indoor Environmental Quality – A Review. *Energy and AI*, 10, 100198. <https://doi.org/10.1016/j.egyai.2022.100198>
- Izonin, I., Tkachenko, R., Mitoulis, S. A., Faramarzi, A., Tsmots, I., & Mashtalir, D. (2023). Machine learning for predicting energy efficiency of buildings: A small data approach. *Procedia Computer Science*, 231, 72-77.
<https://doi.org/10.1016/j.procs.2023.12.173>